

Program-as-Weights: A Programming Paradigm for Fuzzy Functions

Wentao Zhang, Liliana Hotsko, Woojeong Kim, Pengyu Nie, Stuart Shieber, Yuntian Deng

ABSTRACT

Many everyday programming tasks resist clean rule-based implementation, such as alerting on important log lines, repairing malformed JSON, or ranking search results by intent, and are increasingly outsourced to large language model APIs at the cost of locality, reproducibility, and price. We propose fuzzy-function programming: compiling such a function from a natural-language specification into a compact, locally-executable neural artifact. We instantiate this paradigm with Program-as-Weights (PAW), in which a 4B compiler trained on FuzzyBench, a 10M-example dataset we release, emits parameter-efficient adapters for a frozen, lightweight interpreter. A 0.6B Qwen3 interpreter executing PAW programs matches the performance of direct prompting of Qwen3-32B, while using roughly one fiftieth of the inference memory and running at 30 tokens/s on a MacBook M3. PAW reframes the foundation model from a per-input problem solver into a tool builder: invoked once per function definition, it produces a small reusable artifact whose subsequent calls per function application are cheap and offline.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.

Provenance. Ingested by llmXive on 2026-07-04 — scraped from arXiv; via llmXive dashboard, GitHub issue #506; submitted by github-actions[bot].

Source. <https://arxiv.org/abs/2607.02512>

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at <https://github.com/ContextLab/llmXive>.

Automated review. See the accompanying [peer-review-llmxive.pdf](#) for the full reviewer

report.

llmXive follow-up. A separate llmXive study building on this work: [PROJ-991-llmxive-follow-up-extending-program-as-w.](#)

Program-as-Weights: A Programming Paradigm for Fuzzy Functions

Wentao Zhang^{1,*} Liliana Hotsko^{1,*} Woojeong Kim^{2,*}

Pengyu Nie¹ Stuart Shieber³ Yuntian Deng¹

¹University of Waterloo ²Cornell University ³Harvard University

{w564zhan, lhotsko, pynie, yuntian}@uwaterloo.ca

wk247@cornell.edu shieber@seas.harvard.edu

*Equal contribution

Abstract

Many everyday programming tasks resist clean rule-based implementation, such as alerting on important log lines, repairing malformed JSON, or ranking search results by intent, and are increasingly outsourced to large language model APIs at the cost of locality, reproducibility, and price. We propose *fuzzy-function programming*: compiling such a function from a natural-language specification into a compact, locally-executable neural artifact. We instantiate this paradigm with Program-as-Weights (PAW), in which a 4B compiler trained on FuzzyBench, a 10M-example dataset we release, emits parameter-efficient adapters for a frozen, lightweight interpreter. A 0.6B Qwen3 interpreter executing PAW programs matches the performance of direct prompting of Qwen3-32B, while using roughly one fiftieth of the inference memory and running at 30 tokens/s on a MacBook M3. PAW reframes the foundation model from a per-input *problem solver* into a *tool builder*: invoked once per function definition, it produces a small reusable artifact whose subsequent calls per function application are cheap and offline.

1 Introduction

Programming has historically been about writing explicit rules in a formal language designed for the purpose, a programming language. A function is defined by code, and the computer executes it deterministically. For many tasks, this paradigm works beautifully: sorting numbers, processing structured data, computing matrix products. Yet a large class of real-world functions resists precise specification. Consider, for instance, filtering a computer log to alert someone only on the log lines or messages that matter, repairing malformed JSON, or ranking search results by intent. Even apparently “simple” tasks, such as writing a regular expression to parse text with many edge cases, prove brittle. Beyond underspecification, real-world inputs are noisy: typos and format drift routinely break hand-written rules and regexes. These are *fuzzy functions* [Rubio Manzano, 2012]: problems that humans find intuitive but that cannot be fully captured by crisp symbolic rules.

Today, developers frequently outsource such fuzziness to LLM APIs. It is increasingly common to find codebases where a remote LLM is called (e.g., `gpt(“extract answer”, text)`) to implement functions that are otherwise intractable to program. This approach is undeniably convenient, but it is costly, fragile, undermines reproducibility because providers may silently update their models [Kim et al., 2023], and prevents software from being self-contained.

We propose a different paradigm with three steps: the developer *describes* the function in natural language; a neural *compiler* turns that description into a small neural binary; and a frozen, lightweight

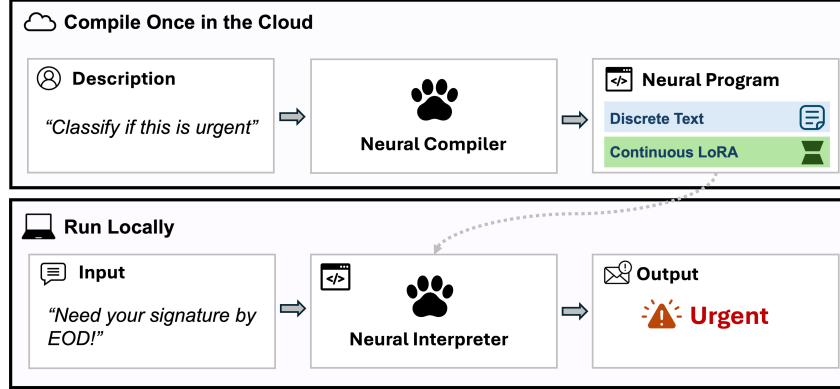


Figure 1: **Overview of the Program-as-Weights paradigm.** *Top: compile once in the cloud.* A natural-language description of a fuzzy function (here, “classify if this is urgent”) is fed to a neural compiler, which produces a neural program. *Bottom: run locally.* A small frozen neural interpreter loads the compiled program and runs the user’s input (“Need your signature by EOD!”) to produce the output (“urgent”). The compiled program is a single file that can be cached, version-controlled, and called offline like any other library function.

neural *interpreter*, installed once on the user’s device, *runs* that binary just like a user-defined function (Figure 1). We call this paradigm Program-as-Weights (PAW). Any sufficiently expressive parameter-efficient module (PEFT) emitted by a hypernetwork can serve as the program form; we instantiate two, prefix-tuning and text-to-LoRA, and find LoRA better, with future PEFTs possibly better still.

A PAW program has two halves. The first is a *pseudo-program* in natural language, a restatement of the user’s specification. The second is a PEFT module that re-tunes the frozen interpreter for this one task: in our precursor system this was a prefix-tuning KV cache; in our current system it is a LoRA generated by the compiler from its own hidden states and injected into the interpreter. The discrete half shields the interpreter from typos and ambiguity in the original specification; the continuous half supplies the fine-grained behavioral control that text alone cannot.

The compile pipeline has two stages, both running 4B Qwen3 models. The first stage is a *pseudo compiler*, an off-the-shelf model we never train: prompted with a small task-rewriting template, it turns the user’s spec into a clean pseudo-program of a paraphrased description plus a handful of input-output examples. The second stage is a *LoRA compiler* that we train: it reads the spec and the pseudo-program and emits the LoRA. We train the LoRA compiler on FUZZYBENCH, a 10M-example dataset we release with this paper, built incrementally across 29 thematic versions covering more than 800 categories of fuzzy text tasks such as classification, format conversion, parsing, fuzzy matching, natural-language commands, agentic tool use, and many more.

The result is a small, fast, and accurate system. A Qwen3-0.6B interpreter executing PAW programs outperforms direct prompting of Qwen3-32B (73.78% vs. 68.70% exact match) at roughly one fiftieth the inference memory. Quantized, the same system runs at 30 tokens per second on a MacBook M3 from a ~430 MB GGUF base shared across functions plus a 23 MB per-program LoRA adapter; a smaller GPT-2 path runs entirely client-side in the browser via WebAssembly.

We see Program-as-Weights as a concrete step toward a small-model future [Belcak et al., 2025], in which the heavy lifting happens once at compile time and the day-to-day work of running software happens locally. We illustrate its applications in five case studies: *output triage* (event-driven log monitoring), *custom classification* (intent-based site navigation), *fuzzy search* (semantic search reranking), *agent preprocessing* (a tool-calling pipeline that scores 93% on TOOLCALL-15), and *creative generation* (a multilingual word-guessing game). Each is the kind of fuzzy task that resists symbolic implementation but does not need an API call to a 30B-parameter model on every input. We additionally show the abstraction’s modality generality: replacing only the compiler with a vision-language model while keeping the same interpreter runs PAW programs on image-conditioned fuzzy tasks. Our code can be found at <https://github.com/programasweights> and a public demo is available at <https://programasweights.com>.

2 Programs as Weights

Let $f : X \rightarrow Y$ denote a function whose behavior is more naturally specified through natural language, examples, or constraints than through symbolic code, a *fuzzy function*. Instead of repeatedly invoking an LLM to approximate f , we propose to compile a *neural program* that specializes a fixed model to implement f .

Formally, let s denote a user specification, expressed in natural language and optionally accompanied by example input-output pairs (x, y) . A neural Compiler maps s to a program p . A small fixed neural Interpreter executes p on inputs $x \in X$ to produce outputs $\hat{y} \in Y$:

$$p = \text{Compiler}(s), \quad \hat{y} = \text{Interpreter}(p, x) \approx f(x). \quad (1)$$

This division mirrors classical programming, where a compiler translates source code into an executable that is later run by a runtime. The crucial difference is that the executable here is a learned parameter blob, and the runtime is a neural network. The interpreter does not need to be retrained: introducing a new fuzzy function only requires compiling a new program p .

Hybrid programs. For conceptual simplicity, p may be viewed as a single continuous object. In our concrete instantiation, however, p is a hybrid of a discrete and a continuous component:

$$p = (p_{\text{discrete}}, p_{\text{continuous}}). \quad (2)$$

The discrete component p_{discrete} is a variable-length sequence of tokens that acts as a self-contained “pseudo-program” presented to the interpreter as part of its input. The continuous component $p_{\text{continuous}}$ can be implemented using any PEFT method, such as a LoRA injected into the interpreter.

Why “program”? This framing matters because it determines how the artifact is used. A compiled PAW program is a single file (~ 23 MB at $Q4_0$ for a 0.6B interpreter, plus a one-time shared base) that can be saved, version-controlled, distributed via package managers, and called from Python or JavaScript with a two-line API. PAW programs are objects of the same kind as Python modules: they have a name and a version, but their behavior is encoded in weights rather than in source code. The compiler is the part that does the heavy lifting; the interpreter is a fixed runtime, comparable to a CPU or a byte-code interpreter in conventional software stacks.

3 The Compiler–Interpreter System

3.1 Compiler–interpreter abstraction

The PAW pipeline has three components, none of which depend on the specific PEFT chosen for the program form. A *pseudo compiler* C_p reads the spec s and produces a discrete pseudo-program p_{discrete} . A *PEFT compiler* C_{PEFT} reads the spec together with p_{discrete} and emits a small parameter-efficient module $p_{\text{continuous}}$ from its hidden states. The frozen *interpreter* ingests $p_{\text{continuous}}$ at runtime — by attaching it to the appropriate target modules and running the user’s input x through it — to produce the output $\hat{y}$. We instantiate the PEFT module in two ways: a prefix-tuning KV cache (Section 3.3) and a LoRA (Section 3.2), with the latter being our current best.

Pseudo compiler. The pseudo compiler C_p is an off-the-shelf Qwen3-4B-Instruct-2507 model that we never train. Given a specification s , we prompt C_p with a small task-rewriting template (full text in Appendix C) that asks for a clean restatement of the task plus a handful of representative input-output examples. The output is the discrete component p_{discrete} of the program.¹ The pseudo compiler is shared by both PEFT instantiations below.

3.2 Text-to-LoRA: our current best

LoRA compiler. The LoRA compiler C_L is a second 4B Qwen3 model, initialized from the same checkpoint as C_p but *trained* (Section 4). Given the spec s and the pseudo-program p_{discrete} produced

¹In an early prototype we trained a single compiler to generate this discrete component via reinforcement learning, but observed that, regardless of seed prompt, the compiler converged on this same paraphrase-plus-examples format; we therefore eliminate the RL stage by directly hand-crafting a prompt that produces this format from an off-the-shelf model.

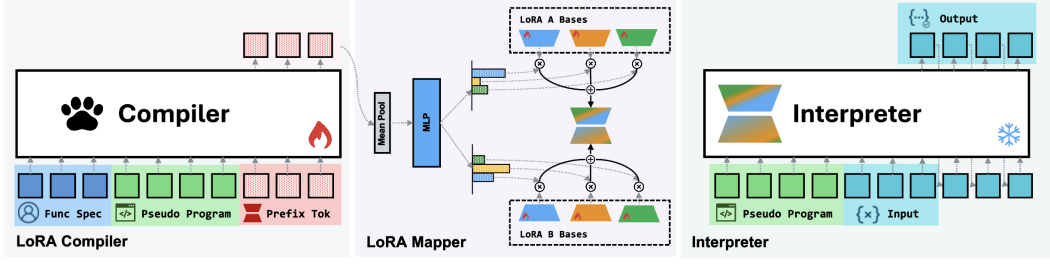


Figure 2: **Text-to-LoRA instantiation of PAW (Section 3.2).** *Left.* The trained LoRA compiler reads the function specification, the pseudo-program produced by an off-the-shelf prompted pseudo compiler C_p (not depicted), and a fixed sequence of learned prefix tokens; it emits prefix-position hidden states H . *Middle.* The LoRA mapper mean-pools H , passes it through an MLP, and projects into mixing coefficients that compose LoRA matrices ($A^{\text{ex}}, B^{\text{ex}}$) over shared learnable bases (eq. (3)). *Right.* The frozen interpreter ingests p_{discrete} prepended to the user input x , with the LoRA hot-attached, and generates the output autoregressively. The same pipeline holds for the prefix-tuning precursor (Section 3.3, with architecture in Figure 18); only the mapping from compiler hidden states to PEFT module changes (LoRA $\rightarrow$ KV-cache mapper).

by C_p , C_L runs a single forward pass on the concatenation $[s \mid p_{\text{discrete}} \mid \text{EOS} \mid \tau_1, \dots, \tau_T]$, where $\tau_{1:T}$ is a fixed sequence of $T = 64$ learned “prefix” tokens. We extract prefix-position hidden states from L compiler layers spaced uniformly by depth ratio (one per interpreter layer), and stack them into the tensor $H \in \mathbb{R}^{L \times T \times d_{\text{teacher}}}$ that is fed to the LoRA mapper (Figure 2).

LoRA mapper. The LoRA compiler’s hidden states H are converted into per-example LoRA weights by a small parameter-efficient module, the *LoRA mapper*. For each interpreter target-module type m (attention $q/k/v/o$ and MLP gate/up/down), the mapper maintains shared learnable bases

$$A_{1:N}^{(m)} \in \mathbb{R}^{N \times r \times d_{\text{in}}^{(m)}}, \quad B_{1:N}^{(m)} \in \mathbb{R}^{N \times d_{\text{out}}^{(m)} \times r}.$$

These hidden states are mean-pooled over both the L depth-aligned layers and the T prefix positions, $\bar{h} = \frac{1}{LT} \sum_{l,t} H_{l,t}$, passed through a shallow MLP trunk ϕ , and projected into mixing coefficients $\alpha_{l,m,n}^{A,B} \in \mathbb{R}$ for each layer l , module type m , and basis index n , via a single linear head. The LoRA at layer l and module m is

$$A_{l,m}^{\text{ex}} = \sum_{n=1}^N \alpha_{l,m,n}^A A_n^{(m)}, \quad B_{l,m}^{\text{ex}} = \sum_{n=1}^N \alpha_{l,m,n}^B B_n^{(m)}. \quad (3)$$

We use rank $r = 64$ and $N = 64$ shared bases per module type, applied to all layers and module types. Per fuzzy function, this injects approximately 38.5M LoRA parameters into the interpreter.²

Interpreter. The interpreter is a frozen language model. To execute a PAW program on input x , we (i) attach the LoRA in eq. (3) to the appropriate target modules, (ii) prepend p_{discrete} to the input x , and (iii) generate the output autoregressively. Because the interpreter is frozen and the LoRA hot-swappable, a single device-resident interpreter can serve unboundedly many PAW programs; Figure 19 illustrates this “one runtime, many programs” picture for three example specifications.

3.3 Prefix-tuning: a precursor instantiation

Prefix compiler. Our precursor system replaced the LoRA mapper with a *prefix-tuning mapper*. The prefix compiler C_P is a second 4B Qwen3 model trained the same way as C_L , with the only difference being how its prefix-position hidden states are consumed. Given $[s \mid p_{\text{discrete}} \mid \text{EOS} \mid \tau_{1:T}]$, C_P produces hidden states $H \in \mathbb{R}^{L \times T \times d_{\text{teacher}}}$ at the same L depth-aligned layers as in Section 3.2. Instead of pooling and projecting into LoRA weights, a small linear mapper ψ projects these hidden states position-wise into KV pairs $(K_{l,t}^{\text{ex}}, V_{l,t}^{\text{ex}}) \in \mathbb{R}^{2 \times d_{\text{int}}}$ that are prepended to the interpreter’s

²We compare this design to several more-expressive alternatives (per-position aggregation, per-layer bases, per-position with per-layer bases, LoRA with prefix-tuning) in Section 7; none improve over the simple shared-basis design.

attention KV cache at every layer, in the manner of standard prefix-tuning [Li and Liang, 2021] (see Figure 18 in the appendix for an architecture diagram). The interpreter then runs x through its frozen attention with the additional T prefix-position keys and values visible to every query.

Both methods solve the task. At a controlled comparison scale (same amount of training compute), the prefix-tuning instantiation reaches 50.4% exact match on FuzzyBench, while the LoRA instantiation reaches 56.5% at $r=18$ ($r=18$ matches the prefix-tuning program size) and 65.7% at $r=64$ (Table 1). Both outperform the no-compiler prompting baseline (9.8%). LoRA is the stronger PEFT and is the instantiation we scale to the full training data in subsequent experiments.

Table 1: **Two PEFT instantiations.** Both methods outperform the prompting baseline.

Method	Accuracy
Prompting	0.098
Prefix Tuning	0.504
Text-to-LoRA, $r=18$	0.565
Text-to-LoRA, $r=64$ (default)	0.657

4 Training

Only the PEFT compiler is trained. The pseudo compiler C_p is held off-the-shelf and frozen; the interpreter is also frozen. The PEFT compiler is trained to produce a PEFT adapter that, when injected into the frozen interpreter alongside a fixed pseudo-program, maximizes the likelihood of the target output. With both endpoints frozen, this reduces to a single supervised objective. We concentrate below on the LoRA instantiation; the prefix-tuning precursor (Section 3.3) was trained with the same SFT recipe described below, substituting the prefix-tuning mapper for the LoRA mapper.

Objective. For each training triple (s, x, y) , we look up a pre-generated pseudo-program $p_{\text{discrete}} = C_p(s)$, run a forward pass through C_L on $[s \mid p_{\text{discrete}} \mid \text{EOS} \mid \tau_{1:T}]$ to obtain prefix-position hidden states, pass those through the LoRA mapper to obtain p_{LoRA} , and inject the result into the interpreter. The loss is the negative mean-token log-likelihood of the target y under the frozen interpreter:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,x,y)} \left[-\log P_\phi(y \mid p_{\text{discrete}}, p_{\text{LoRA}}(\theta; s, p_{\text{discrete}}), x) \right], \quad (4)$$

where θ is the parameters of C_L and the LoRA mapper, and ϕ are the interpreter parameters. The gradient flows back through the frozen interpreter into the LoRA mapper and from there into C_L ’s hidden states. Full hyperparameters and compute setup are in Appendix G.

5 FuzzyBench: A 10M-Example Dataset of Fuzzy Functions

A central obstacle to training PAW-style methods is the lack of a public dataset for “compile a fuzzy function from a specification.” We construct **FuzzyBench**, a 10M-example dataset in which every example is a triple (s, x, y) of (specification, input, target output), generated using gpt-5.2.

Construction. We use a two-stage pipeline. In the first stage, we prompt gpt-5.2 to generate natural-language specifications of fuzzy functions. Each prompting call produces eight specifications, and we run repeated calls under different category constraints to cover the breadth of fuzzy tasks developers actually encounter. In the second stage, for each specification, we prompt gpt-5.2 again to generate eight input/output pairs. Specifications are split 80/10/10 by spec into train/validation/test, so that test specifications are entirely unseen at training time. For evaluation, we construct a *verified* test set on which an independent strong model (gpt-5-mini) and gpt-5.2 agree on the output, removing examples where the target itself is ambiguous. Full prompts are in Appendix B.

Thematic coverage. FuzzyBench is built incrementally over 29 versions, each adding 100K-500K examples covering a new family of fuzzy tasks. Figure 3 groups the resulting 10M examples into seven high-level task families that span what developers actually encounter in fuzzy logic: from raw text processing and parsing to agentic tool use, web intelligence, code-and-command generation, and

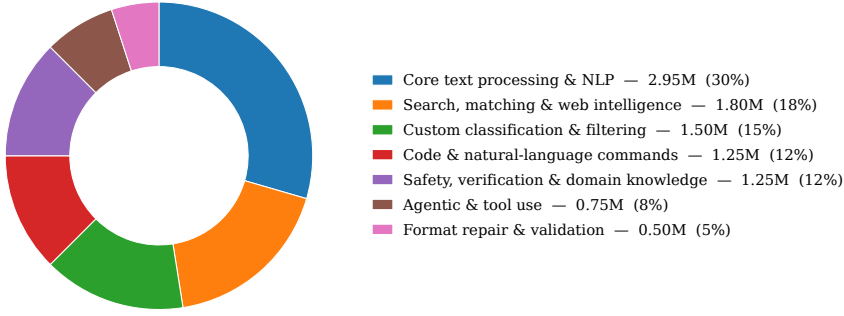


Figure 3: **FuzzyBench-10M task-family distribution.** 29 incremental thematic versions are mapped to 7 high-level families. “Core text processing & NLP” is the largest family because the v1 base layer (2.5M examples; 277 base categories) covers parsing, classification, NER, coreference, and sentiment; the remaining 7.5M examples spread across the other six families.

safety/verification. The full per-version timeline (29 entries; the first version alone establishes 277 base categories, and the final dataset covers more than 800 sub-categories) is in Appendix F.

Noise variants. For robustness evaluation, we additionally release noise-perturbed versions of the test set along eight axes: typos, grammar errors, ambiguity, formatting drift, “all noise” (combined), terse phrasing, casual phrasing, and paraphrase. Each axis comes in three intensity levels (light, medium, heavy). Section 8 reports robustness numbers.

Empirical ceiling. The data-generating model itself, gpt-5.2, achieves 96.09%; gpt-5-mini achieves 91.87%. These bound how high any compiled function trained on this data can reach.

6 Main Results

We compare PAW against three families of baselines, all evaluated on the same test sets as PAW so that any compute or data-generation differences are absorbed in the comparison.

Baselines. (i) *Direct prompting* of open-weight LMs (Qwen3 0.6B, 4B, 8B, 14B, 32B; OLMo3-7B; gpt-oss-20B), and of two API models that bound the empirical ceiling (gpt-5-mini and gpt-5.2). (ii) *Symbolic code generation*: ALCHEmist’s LM-to-code pipeline [Huang et al., 2024b], where a strong LM writes Python code to solve the fuzzy task and the code is executed at inference. (iii) *Standard adaptation of the same 0.6B base*: full fine-tuning across 1-4 epochs, and fixed (non-compiler-generated) LoRAs at ranks $r \in \{18, 64, 128\}$.

Main result. Table 2 summarizes the main numbers. A 0.6B-parameter interpreter executing PAW programs achieves **73.78%** exact match on FuzzyBench, outperforming prompting Qwen3-32B (68.70%) while using approximately $50\times$ less inference memory (~ 1.2 GB at bf16 vs. ~ 60 GB).

Cross-interpreter scaling. Among three interpreters GPT-2 124M, Qwen3 0.6B, and Qwen3.5 0.8B, Qwen3 0.6B is the strongest interpreter; the hybrid 0.8B is slightly weaker. GPT-2 124M, despite having only 1/5 the parameters of Qwen3 0.6B and no instruction tuning, still achieves 54%, suggesting that the compiler-generated LoRA can encode usable task adaptations even into very small, weakly-capable bases.

Multimodal generalization without changing the interpreter. The compiler-interpreter abstraction extends to image-conditioned fuzzy functions *without* changing the interpreter. We swap the text-only Qwen3-4B-Instruct compiler for the same-family Qwen3-VL-4B compiler [Bai et al., 2025], keep the same Qwen3 0.6B interpreter, and reuse the same LoRA mapper. Image conditioning is fully encoded in the PEFT module emitted by the VL compiler, so the small text interpreter never sees

Table 2: **Main results.** FuzzyBench uses exact match accuracy on the verified test set. Following the WRENCH benchmark setup of ALChemist [Huang et al., 2024b], SMS uses F1 and the rest use Acc. *Contained* indicates the program is self-contained and executable without internet access. PS is per-program shipping size; for prompting baselines this is the prompt/spec size, and for PAW it is the deployed PEFT adapter (*Q4_0* quantized for Qwen3 0.6B and Qwen3.5 0.8B; fp32 for GPT-2). [†]: numbers taken from Huang et al. [2024b], which uses 10-sample majority voting; the reimplementation row uses single-sample for fairness. *: zero F1 due to zero recall.

Method	Contained	Interp. Size	FuzzyBench		YouTube		SMS		Yelp		IMDB	
			Acc	PS	Acc	PS	F1	PS	Acc	PS	Acc	PS
gpt-5.2 (API)	×	–	96.09%	0.73 KB	95.20%	0.95 KB	97.06%	1.03 KB	98.55%	1.69 KB	95.60%	2.25 KB
gpt-5-mini (API)	×	–	91.87%	0.73 KB	93.60%	0.95 KB	91.03%	1.03 KB	98.13%	1.69 KB	94.96%	2.25 KB
Local LM (Qwen3 0.6B)	✓	0.6B	9.84%	0.73 KB	52.80%	0.95 KB	0.00%*	1.03 KB	89.55%	1.69 KB	66.88%	2.25 KB
Local LM (Qwen3 4B)	✓	4B	49.63%	0.73 KB	90.80%	0.95 KB	92.54%	1.03 KB	97.53%	1.69 KB	93.76%	2.25 KB
Local LM (Qwen3 8B)	✓	8B	52.15%	0.73 KB	94.40%	0.95 KB	91.55%	1.03 KB	97.95%	1.69 KB	94.52%	2.25 KB
Local LM (Qwen3 14B)	✓	14B	63.96%	0.73 KB	93.20%	0.95 KB	92.75%	1.03 KB	97.74%	1.69 KB	92.64%	2.25 KB
Local LM (Qwen3 32B)	✓	32B	68.70%	0.73 KB	93.60%	0.95 KB	89.04%	1.03 KB	98.11%	1.69 KB	94.64%	2.25 KB
Local LM (OLMo3 7B)	✓	7B	39.84%	0.73 KB	90.00%	0.95 KB	90.14%	1.03 KB	97.66%	1.69 KB	93.28%	2.25 KB
Local LM (gpt-oss-20B)	✓	20B	85.45%	0.73 KB	91.60%	0.95 KB	89.05%	1.03 KB	97.42%	1.69 KB	92.08%	2.25 KB
LM→Code [Huang et al., 2024b]	✓	29 MB	–	–	89.10% [†]	–	90.00% [†]	–	57.50% [†]	–	66.20% [†]	–
LM→Code (Reimplementation)	✓	29 MB	35.81%	0.08 KB	70.46%	0.09 KB	86.41%	0.06 KB	50.35%	0.05 KB	73.92%	0.08 KB
PAW (Qwen3 0.6B)	✓	0.6B	73.78%	23 MB	90.40%	23 MB	80.77%	23 MB	95.82%	23 MB	90.64%	23 MB
PAW (Qwen3.5 0.8B)	✓	0.8B	67.29%	16 MB	88.40%	16 MB	84.55%	16 MB	94.05%	16 MB	82.68%	16 MB
PAW (GPT-2 124M)	✓	124M	54.39%	38 MB	93.60%	38 MB	77.50%	38 MB	93.16%	38 MB	82.12%	38 MB

Table 3: **Image-conditioned fuzzy functions.** The PAW rows use the same Qwen3 0.6B and Qwen 3.5 0.8B interpreters as in Table 2 (only the compiler is swapped, from Qwen3-4B-Instruct to Qwen3-VL-4B).

Method	Interp. Size	Circuit	Chemical	Music	Im2SMILES	Im2LaTeX	TextVQA
AndesVL 0.6B [AndesVL Team, OPPO AI Center, 2025]	0.6B	0.183	0.214	0.448	0.000	0.435	0.718
Qwen3-VL 2B-Instruct [Bai et al., 2025]	2B	0.186	0.258	0.470	0.016	0.408	0.836
Qwen3-VL 4B-Instruct [Bai et al., 2025]	4B	0.196	0.221	0.450	0.044	0.399	0.822
PAW prefix-tuning (Qwen3 0.6B)	0.6B	0.241	0.365	0.525	0.175	0.391	0.612
PAW LoRA (Qwen3 0.6B)	0.6B	0.274	0.414	0.552	0.203	0.181	0.721
PAW LoRA (Qwen3.5 0.8B)	0.8B	0.284	0.438	0.573	0.285	0.204	0.755

pixels. Table 3 reports six image-conditioned tasks: three CoSyn-400K diagram-understanding tasks (Chemical, Circuit, Music) [Yang et al., 2025, Deitke et al., 2024], the structured-output Im2LaTeX-100K [Deng et al., 2017] and Im2SMILES-20K [Deng et al., 2023] tasks, and the open-ended visual question answering TextVQA [Singh et al., 2019]; full prompts are in Appendix D.

PAW (LoRA) outperforms all VLM baselines (up to 4B parameters) on the three CoSyn diagram tasks (Circuit 0.274 vs. 0.196 best baseline; Chemical 0.414 vs. 0.258; Music 0.552 vs. 0.470) at ~0.6B interpreter size. On the long-form structured generation task Im2LaTeX, PAW (LoRA) is weaker than its prefix-tuning precursor (0.181 vs. 0.391); a discrete-pseudo-only ablation in Appendix D.1 shows the gap arises because the long input/output examples in the pseudo-program crowd the small interpreter’s context budget on long-form tasks.

7 Ablations

Architectural variants of the LoRA mapper. We tried several variants of the LoRA mapper that, on paper, are strictly more expressive than the default (mean-pool over prefix tokens, shallow trunk, shared bases). Each made things worse. Table 4 reports accuracy across these variants. The simplest design of mean-pooling the prefix-token hidden states into one vector, running a single residual MLP, and projecting to mixing coefficients over a shared basis set, is the strongest. We do not have a clean theoretical explanation for this; we report the finding so that future work need not rediscover it.

Compiler vs. no compiler. Table 5 compares PAW with three same-base “no compiler” baselines on FuzzyBench: full fine-tuning of the 0.6B Qwen3 interpreter, and per-task fixed LoRAs at three ranks. The same data, the same base model, the same training budget; only the compiler is removed. PAW exceeds full fine-tuning by 15.4 percentage points and the strongest fixed LoRA by 21.7 points, showing that the gain comes specifically from compiler-generated LoRA.

Table 4: **Architectural variants of the LoRA mapper.** “More expressive” design choices that we expected to help all underperformed the simple default.

Mapper variant	Accuracy
Default ($r=64$, $N=64$, shared bases)	0.6223
Per-position aggregation	0.5598
Per-position + per-layer bases	0.5559
Per-layer bases (only)	0.6028
LoRA + prefix-tuning (both pathways)	0.6033

Table 5: **No compiler baselines.** PAW row reproduced from Table 2 for comparison.

Method (0.6B base)	Accuracy
Fixed LoRA $r=18$	0.4236
Fixed LoRA $r=64$	0.5210
Fixed LoRA $r=128$	0.5159
Full fine-tuning	0.5840
PAW (Qwen3 0.6B)	0.7378

Table 6: **Robustness to noise.** The 8-axis variants modify the spec but leave the input unchanged. PAW degrades only slightly even under combined heavy noise.

	clean	typos	grammar	ambiguity	formatting	all-noise	terse	paraphrase
PAW Qwen3 0.6B (e2)	0.6692	0.6621	0.6731	0.6511	0.6526	0.6326	0.6499	0.6614
drop from clean	–	−0.7%	+0.4%	−1.8%	−1.7%	−3.7%	−1.9%	−0.8%

Table 7: **The pseudo-program protects the interpreter from noisy specifications.** On heavy-typo specifications, feeding raw spec is 4.5 points worse than feeding the pseudo-program to the interpreter.

Interpreter input	Accuracy (clean)	Accuracy (heavy typos)
Pseudo-program (default)	0.6443	0.6108
Raw spec	0.6285	0.5662

Other ablations. Additional ablations on model architecture decisions can be found at Appendix H.

8 Robustness to Noisy Specifications

Real specifications written by developers are noisy: they contain typos, ambiguity, and grammar errors. We evaluate PAW on noise-perturbed versions of the test_clean specifications across seven axes (typos, grammar, ambiguity, formatting, all-noise combined, terse, paraphrase).

Table 6 reports the robustness results. PAW degrades only slightly under heavy noise — but *why*? We hypothesise that this robustness is mediated by the discrete pseudo-program: the 4B compiler converts the noisy spec into a clean restatement before the small interpreter ever sees it. To test the hypothesis, we trained a variant that bypasses the pseudo-program and feeds the raw spec s directly to the interpreter. The result (Table 7) confirms the hypothesis. On clean inputs, feeding the pseudo-program rather than the raw spec is only 1.6 points better. On *heavy-typo* specifications, however, the gap widens to 4.5 points. The compiler, a 4B LM whose entire job is to read fuzzy specifications and emit a clean restatement, effectively denoises the input that the small interpreter sees, which is why PAW degrades little when the original specification is corrupted.

9 Local Execution

Beyond benchmarks, to make PAW practical to use, we built a developer interface.

Developer interface. A PAW program is a single file that can be downloaded, cached, and called via a small Python or JavaScript API. Figure 4 shows a complete minimal Python example: `paw.compile(prompt)` sends a specification to a compiler service and returns a serializable program object; `paw.function(id_or_path)` loads a compiled program and exposes it as a Python callable. After the first download, all execution happens locally with no external API calls.

Quantization without measurable accuracy loss. On-device execution requires a small footprint. We quantize both the shared interpreter base and each per-program LoRA adapter to GGUF formats compatible with `llama.cpp` [Gerganov and llama.cpp contributors, 2023]. Table 8 reports our

```

import programasweights as paw

# (1) Describe the fuzzy function
spec = """Classify if this email
requires immediate attention.
""".strip()

# (2) Compile to a neural program
program = paw.compile(spec, \
    slug="email-triage")

```

Listing 1: Compile a fuzzy function

```

import programasweights as paw

# (3) Load the compiled program
fn = paw.function("email-triage")

# (4) Call it like a function
print(fn("Thesis defense moved "
        "to 3pm; need your "
        "signature today."))

# -> "immediate"

```

Listing 2: Run the compiled program locally

Figure 4: **Developer interface.** *Left:* the compiler translates a natural-language specification into a neural program. *Right:* the interpreter loads this program and exposes it as a local function.

Table 8: **Quantization on the 0.6B Qwen3 interpreter.** A $Q6_K$ base + $Q4_0$ LoRA is indistinguishable from bf16 within noise; $Q4_K_M$ loses 1.3 points but cuts total disk to ~ 507 MB.

Configuration	Base size	Adapter size	Accuracy
PyTorch bf16 (no quantization)	1515 MB	–	0.6580
fp16 base + fp32 LoRA	1509 MB	162 MB	0.6594
$Q8_0$ base + $Q4_0$ LoRA	805 MB	23 MB	0.6567
$Q6_K$ base + $Q4_0$ LoRA	623 MB	23 MB	0.6575
$Q5_K_M$ base + $Q4_0$ LoRA	551 MB	23 MB	0.6477
$Q4_K_M$ base + $Q4_0$ LoRA	484 MB	23 MB	0.6453
$IQ4_XS$ base + $Q4_0$ LoRA	430 MB	23 MB	0.6462

quantization findings on the 0.6B Qwen3 interpreter, validated at a 4096-example test subset: a 4-bit base ($Q4_K_M$, ~ 484 MB) plus a $Q4_0$ LoRA adapter (~ 23 MB per program) loses only 1.3 points relative to bf16, and a $Q6_K$ base plus $Q4_0$ adapter is statistically indistinguishable from bf16.

Latency on a MacBook M3. On a MacBook M3 with Metal acceleration, the $Q5_K_M$ base + $Q4_0$ adapter runs at 31.6 tokens/s with a 0.48 s cold load. Full per-quant tables for GPT-2 124M and Qwen3.5 0.8B are in Appendix K.

Case studies. We applied PAW to five use cases. *Event-driven log monitoring* replaces the naive wait-based terminal watching in Cursor with a local classifier that fires only on the lines that matter. *Intent-based site navigation* provides a natural-language quick-find for a website without an LLM API call per request. *Semantic search reranking* adds intent-aware fuzzy search to an existing keyword index, again without putting an LLM in the request path. For *tool calling*, a 10-PAW-function pipeline scores 93% on TOOLCALL-15, capturing tool-routing behavior usually reserved for much larger models. The *multilingual word-guessing game* (Alien-Taboo) is a fuzzy interactive game in which each player turn is served by a 0.6B PAW interpreter on a small server, with one PAW program per language; the LLM is invoked only at compile time, which is what makes a game of this kind economical to host. Full details are in Appendix M.

10 Related Work

Hypernetworks. Hypernetworks [Ha et al., 2017] generate the weights of a target network from a small embedding, originally for vision and language modeling; subsequent work used them for continual learning [von Oswald et al., 2020], multi-task NLP via shared hypernetworks across tasks and layers [Karimi Mahabadi et al., 2021b], and as parameter-efficient adapters in their own right [Karimi Mahabadi et al., 2021a]. The text-conditioned subfamily relevant to PAW maps a natural-language task description to PEFT parameters in a single forward pass: Hypter [Ye and Ren, 2021] generates BART-Large adapters from descriptions; HINT [Ivison et al., 2023] generates prefix-and-adapter modules from instructions to amortize per-query encoding cost; HyperTuning [Phang et al., 2023] introduces a T5-based hypermodel that emits soft prefixes or LoRA parameters from

few-shot examples; Text-to-LoRA [Charakorn et al., 2025] maps textual task descriptions to LoRAs distilled from pre-trained adapters; Generative Adapter [Chen et al., 2025] produces task-specific adapters from a single forward pass over context; HyperSteer [Sun et al., 2025] extends the idea to activation steering; Gist [Mu et al., 2023] compresses prompts into a few prefix tokens via attention-mask training; and MEND [Li et al., 2024] distills demonstrations into vectors via two-stage meta-distillation. The most recent work closest to PAW maps natural-language *contexts* to LoRA in a single forward pass: SHINE [Liu et al., 2026] as a scalable in-context hypernetwork; HypeLoRA [Trojan and Geßala, 2026] for calibrated PEFT generation with structural coupling across layers; Doc-to-LoRA [Charakorn et al., 2026], which meta-learns to internalise a document into a LoRA adapter that the base model can then query without re-consuming the context; and Latent Context Compilation [Li et al., 2026], which explicitly frames a LoRA module as a *compiler* that distills long context into compact portable buffer tokens. LoRA composition methods such as LoraHub [Huang et al., 2024a] share basis sets across tasks, parallel to our shared-basis LoRA mapper. Compared with these, PAW (a) emits a hybrid (discrete pseudo-program + continuous PEFT) program rather than a continuous-only adapter; (b) is trained on *programmer-style fuzzy-function specifications* (FuzzyBench-10M’s 800+ task families) rather than on QA contexts or distilled per-task adapters; and (c) targets a developer-facing API where the compiled program is a versioned, distributable software artifact.

Parameter-efficient fine-tuning. The PEFT building blocks our compiler emits are well-studied. Adapters [Houlsby et al., 2019, Pfeiffer et al., 2021] insert small trainable bottlenecks into a frozen backbone; prefix-tuning [Li and Liang, 2021] prepends learned key-value pairs to attention; prompt tuning [Lester et al., 2021, Liu et al., 2022b] learns soft input embeddings; LoRA [Hu et al., 2022] learns low-rank decomposed updates to the linear projections of attention and MLP layers; AdaLoRA [Zhang et al., 2023] dynamically allocates rank budgets across layers; DoRA [Liu et al., 2024] decomposes pre-trained weights into magnitude and direction and applies LoRA only to the direction component, closing the gap to full fine-tuning; QLoRA [Dettmers et al., 2023] combines quantization with LoRA for memory-efficient fine-tuning. PAW differs in that the PEFT module is *generated per example by a separate compiler from a textual specification*, rather than learned per task by gradient descent on the target task. T-Few [Liu et al., 2022a] argues that PEFT can outperform in-context learning at lower deployment cost — a related framing, with the difference that T-Few learns its PEFT per task while we generate it from a description.

Synthetic instruction-data generation. FuzzyBench-10M is generated by an LLM (gpt-5.2) and follows the methodological precedent of LLM-generated instruction datasets. Self-Instruct [Wang et al., 2023] prompts a strong LLM to generate diverse instruction-input-output triples that are then used to fine-tune a smaller model. Unnatural Instructions [Honovich et al., 2023] similarly generates instructions automatically. Textbooks Are All You Need [Gunasekar et al., 2023] argues for synthetic-textbook-style training data for small-model pre-training. Magpie [Xu et al., 2025] self-synthesises 4M alignment instances from an aligned LLM with no seed prompts. FuzzyBench-10M differs in that the data-generating pipeline is task-class-specific (29 thematic versions covering categories developers actually encounter, rather than open-ended prompts), and we explicitly construct a verified test split (Section 5) where two strong LLMs must agree on the output to filter ambiguous targets. Recent small-model technical reports [Abdin et al., 2024, Gemma Team, 2025] similarly emphasise high-quality synthetic data as the primary lever for closing capability gaps with frontier models at small scales.

Model distillation. ALCHEmist [Huang et al., 2024b] distills labelling logic from LLMs into Python programs that run on a standard interpreter. PAW shares the motivation of amortizing LLM usage but compiles directly into *neural* weights instead of textual code, which lets it implement fuzzy functions that resist symbolic encoding. Binder [Cheng et al., 2023] translates a task input into a SQL/Python program with embedded LM API calls; PAW differs in that the program is the weights themselves, not a piece of text containing API calls. Distilling Step-by-Step [Hsieh et al., 2023] distills LLM reasoning into smaller fine-tuned models with rationales as auxiliary supervision; PAW shares the goal of replacing large-LM inference with small-model inference, but achieves it via per-task compile rather than per-task fine-tuning.

Neural programs. Representing programs in neural networks is a long-standing direction [Graves et al., 2014, Reed and de Freitas, 2016], with some work compiling formal code into network

weights [Weiss et al., 2021, Gruau et al., 1995]. PAW differs in its training and use model: programs are not learned per task but produced on demand by a single compiler, then executed on a fixed interpreter and freely shared. A recent trend argues for replacing API LLMs with small, locally executed models [Belcak et al., 2025, Abdin et al., 2024, Gemma Team, 2025]; PAW is one realization. The crucial difference between PAW and “just use a small LLM” is that the small model’s behaviour is configured per fuzzy function by a compiler rather than baked into a fine-tune. Practical on-device deployment of small models has been driven by post-training quantization (GPTQ [Frantar et al., 2023], AWQ [Lin et al., 2024], QLoRA’s quantization-aware finetuning [Dettmers et al., 2023]) and lightweight inference runtimes (llama.cpp [Gerganov and llama.cpp contributors, 2023], in-browser via wllama [wllama contributors, 2024]); we use these directly. Most recent small-LM technical reports [Abdin et al., 2024, Gemma Team, 2025] also adopt LoRA-flavoured PEFT for multimodal extensions and downstream adaptation, complementing the developer-API direction PAW pursues.

11 Conclusion

We introduced Program-as-Weights, a programming paradigm in which a fuzzy function is compiled once into a small neural binary and executed locally on a fixed interpreter. On FuzzyBench, a 0.6B-parameter interpreter executing PAW programs matches Qwen3-32B prompting at $\sim 50\times$ less inference memory and runs at 30 tok/s on a MacBook M3 with quantized GGUF; we illustrate the paradigm through five case studies. The same abstraction extends to image-conditioned fuzzy tasks by swapping only the compiler for a vision-language model. We hope Program-as-Weights contributes to a future in which small LMs serve as the runtime [Belcak et al., 2025], where large models compile and small models execute, and the role of foundation models shifts from per-input *problem solver* to per-function *tool builder*.

Acknowledgments

We thank Sasha Rush for his guidance and contributions to the earlier project that laid the foundation for this work. We also thank Saarang Agarwal, Austin Dong, Mohammad Jaffer Iqbal, Bihui Jin, Yinxi Li, Jiale Amber Wang, and the anonymous reviewers for their valuable comments and feedback.

This research was supported by a Starter Grant from the University of Waterloo and by the Natural Sciences and Engineering Research Council of Canada (NSERC) under grant numbers RGPIN-2024-04909 and RGPIN-2024-05178. Computational resources were provided by Compute Ontario (computeontario.ca) and the Digital Research Alliance of Canada (alliancecan.ca). We also thank OpenAI’s Research Access Program for providing API credits. Wentao Zhang was supported in part by these sources and by the Dr. Derick Wood Graduate Scholarship, generously funded by Ms. Mary Chen.

References

- Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report, 2024. URL <https://arxiv.org/abs/2412.08905>.
- AndesVL Team, OPPO AI Center. Andesvl technical report: An efficient mobile-side multimodal large language model, 2025. URL <https://arxiv.org/abs/2510.11496>.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025. URL <https://arxiv.org/abs/2511.21631>.

- Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic ai, 2025. URL <https://arxiv.org/abs/2506.02153>.
- Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. Text-to-LoRA: Instant transformer adaption. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=zWskCdu3QA>.
- Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. Doc-to-LoRA: Learning to instantly internalize contexts, 2026. URL <https://arxiv.org/abs/2602.15902>.
- Tong Chen, Hao Fang, Patrick Xia, Xiaodong Liu, Benjamin Van Durme, Luke Zettlemoyer, Jianfeng Gao, and Hao Cheng. Generative adapter: Contextualizing language models in parameters with a single forward pass. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=bc3sUsS6ck>.
- Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=1H1PV42cbF>.
- Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models. *arXiv preprint arXiv:2409.17146*, 2024.
- Yuntian Deng, Anssi Kanervisto, Jeffrey Ling, and Alexander M. Rush. Image-to-markup generation with coarse-to-fine attention. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 980–989. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/deng17a.html>.
- Yuntian Deng, Noriyuki Kojima, and Alexander M Rush. Markup-to-image diffusion models with scheduled sampling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=81VJdmOE2ol>.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.14314>.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.17323>.
- Gemma Team. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Georgi Gerganov and llama.cpp contributors. llama.cpp: Llm inference in c/c++, 2023. URL <https://github.com/ggerganov/llama.cpp>.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014. URL <https://arxiv.org/abs/1410.5401>.
- Frédéric Gruau, Jean-Yves Ratajszczak, and Gilles Wiber. A neural compiler. *Theoretical Computer Science*, 141(1):1–52, 1995. ISSN 0304-3975. doi: [https://doi.org/10.1016/0304-3975\(94\)00200-3](https://doi.org/10.1016/0304-3975(94)00200-3). URL <https://www.sciencedirect.com/science/article/pii/0304397594002003>.
- Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. Textbooks are all you need, 2023. URL <https://arxiv.org/abs/2306.11644>.
- David Ha, Andrew M. Dai, and Quoc V. Le. HyperNetworks. In *The 5th International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1609.09106>.

- Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. Unnatural instructions: Tuning language models with (almost) no human labor. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL <https://arxiv.org/abs/2212.09689>.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. URL <https://arxiv.org/abs/1902.00751>.
- Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023. URL <https://arxiv.org/abs/2305.02301>.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. LoraHub: Efficient cross-task generalization via dynamic LoRA composition. In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024a. URL <https://arxiv.org/abs/2307.13269>.
- Tzu-Heng Huang, Catherine Cao, Vaishnavi Bhargava, and Frederic Sala. The alchemist: Automated labeling 500x cheaper than llm data annotators. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 62648–62672. Curran Associates, Inc., 2024b. doi: 10.52202/079017-2003. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/72802bef5cf1a3449e909b20c2ae18d5-Paper-Conference.pdf.
- Hamish Ivison, Akshita Bhagia, Yizhong Wang, Hannaneh Hajishirzi, and Matthew Peters. HINT: Hypernetwork instruction tuning for efficient zero- and few-shot generalisation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11272–11288, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.631. URL <https://aclanthology.org/2023.acl-long.631/>.
- Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers. In *Advances in Neural Information Processing Systems*, 2021a. URL <https://arxiv.org/abs/2106.04647>.
- Rabeeh Karimi Mahabadi, Sebastian Ruder, Mostafa Dehghani, and James Henderson. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 565–576, Online, August 2021b. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.47. URL <https://aclanthology.org/2021.acl-long.47/>.
- Hyunwoo Kim, Melanie Sclar, Xuhui Zhou, Ronan Bras, Gunhee Kim, Yejin Choi, and Maarten Sap. FANToM: A benchmark for stress-testing machine theory of mind in interactions. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14397–14413, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.890. URL <https://aclanthology.org/2023.emnlp-main.890/>.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021. URL <https://arxiv.org/abs/2104.08691>.

- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.353. URL <https://aclanthology.org/2021.acl-long.353/>.
- Yichuan Li, Xiyao Ma, Sixing Lu, Kyumin Lee, Xiaohu Liu, and Chenlei Guo. MEND: Meta demonstration distillation for efficient and effective in-context learning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=2Y5kBPtU0o>.
- Zeju Li, Yizhou Zhou, and Qiang Xu. Latent context compilation: Distilling long context into compact portable memory, 2026. URL <https://arxiv.org/abs/2602.21221>.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL <https://arxiv.org/abs/2306.00978>.
- Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*, 2022a. URL <https://arxiv.org/abs/2205.05638>.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2402.09353>.
- Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-Tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022b. URL <https://arxiv.org/abs/2110.07602>.
- Yewei Liu, Xiyuan Wang, Yansheng Mao, Yoav Gelberg, Haggai Maron, and Muhan Zhang. SHINE: A scalable in-context hypernetwork for mapping context to LoRA in a single pass, 2026. URL <https://arxiv.org/abs/2602.06358>.
- Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/3d77c6dcc7f143aa2154e7f4d5e22d68-Abstract-Conference.html.
- Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapter-Fusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2021. URL <https://arxiv.org/abs/2005.00247>.
- Jason Phang, Yi Mao, Pengcheng He, and Weizhu Chen. HyperTuning: Toward adapting large language models without back-propagation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 27854–27875. PMLR, 2023. URL <https://proceedings.mlr.press/v202/phang23a.html>.
- Scott Reed and Nando de Freitas. Neural programmer-interpreters, 2016. URL <https://arxiv.org/abs/1511.06279>.
- Clemente Rubio Manzano. Design and implementation of a fuzzy logic programming language using weak unification. *AI Commun.*, 25(4):365–367, October 2012. ISSN 0921-7126.
- Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards VQA models that can read. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8317–8326, 2019.

- Jiuding Sun, Sidharth Baskaran, Zhengxuan Wu, Michael Sklar, Christopher Potts, and Atticus Geiger. Hypersteer: Activation steering at scale with hypernetworks, 2025. URL <https://arxiv.org/abs/2506.03292>.
- Bartosz Trojan and Filip Gębala. HypeLoRA: Hyper-network-generated LoRA adapters for calibrated language model fine-tuning, 2026. URL <https://arxiv.org/abs/2603.19278>.
- Johannes von Oswald, Christian Henning, Benjamin F. Grewe, and João Sacramento. Continual learning with hypernetworks. In *International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1906.00695>.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL <https://arxiv.org/abs/2212.10560>.
- Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers, 2021. URL <https://arxiv.org/abs/2106.06981>.
- llama contributors. llama: Webassembly bindings for llama.cpp, 2024. URL <https://github.com/ngxson/llama>.
- Zhangchen Xu, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin. Magpie: Alignment data synthesis from scratch by prompting aligned LLMs with nothing. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2406.08464>.
- Yue Yang, Ajay Patel, Matt Deitke, Tanmay Gupta, Luca Weihs, Andrew Head, Mark Yatskar, Chris Callison-Burch, Ranjay Krishna, Aniruddha Kembhavi, et al. Scaling text-rich image understanding via code-guided synthetic multimodal data generation. *arXiv preprint arXiv:2502.14846*, 2025.
- Qinyuan Ye and Xiang Ren. Learning to generate task-specific adapters from task description. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 646–653, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-short.82. URL <https://aclanthology.org/2021.acl-short.82/>.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2303.10512>.

A Web Interface for PAW Compilation

We provide a hosted web interface that accepts a fuzzy specification, compiles it, lets the user test it interactively, and exports the compiled program as either a serialized weight file or a program identifier that can be loaded through the Python API. The three steps of the workflow are illustrated in Figures 5, 6, and 7. Compilation runs on a GPU-backed server so that users do not need to provision GPUs locally; once downloaded, the compiled program runs entirely offline on the local interpreter.

The screenshot shows a web interface titled "Program Specification". Below the title is a description: "Describe what your program should do. It can accept text or images as input and will produce text output." There are two buttons: "New Program" (highlighted in blue) and "Load Existing". Below these is a large text input area containing the text: "Extract all email addresses from text and return them as a JSON list". Below the input area is a section titled "Input/Output Examples" with a right arrow and the word "Optional". At the bottom, there is a large purple button labeled "Compile Program" with a star icon, and a smaller button labeled "Models" with a right arrow.

Figure 5: **Step 1: Compile a program from natural language.** The user specifies a fuzzy function in natural language. Image inputs are also supported.

The screenshot shows a web interface titled "Try Your Program". At the top right, there are two buttons: "<> Raw" and "↻ New Test". Below the title is a description: "See how your program handles different inputs. Results appear instantly." There is a section titled "Test 1" with a right arrow. Below this is a table with two columns: "Input" and "Output". The "Input" column has a toggle for "Advanced" and contains the text: "Hi team, Please contact the following people for assistance: - Alice Walker: alice.walker@example.com - Bob (sales): bob-sales@shop.co.uk - Customer support: support@company.org For personal queries reach me at john.doe+personal@gmail.com. You can also CC our lead: lead_dev@sub.domain.io. Thanks!". The "Output" column has a toggle for "JSON" and contains the JSON array: ["alice.w.cur@example.com", "bob-sales@shop.co.uk", "support@company.org", "john.doe+personal@gmail.com", "lead_dev@sub.domain.io"]. Below the input and output areas are two buttons: "Show less" and "Show less". At the bottom, there is a dashed box with a plus icon and the text "Add Test Case".

Figure 6: **Step 2: Interactively test the compiled program.** Users can provide test inputs and inspect the corresponding outputs, enabling rapid validation and refinement before download.

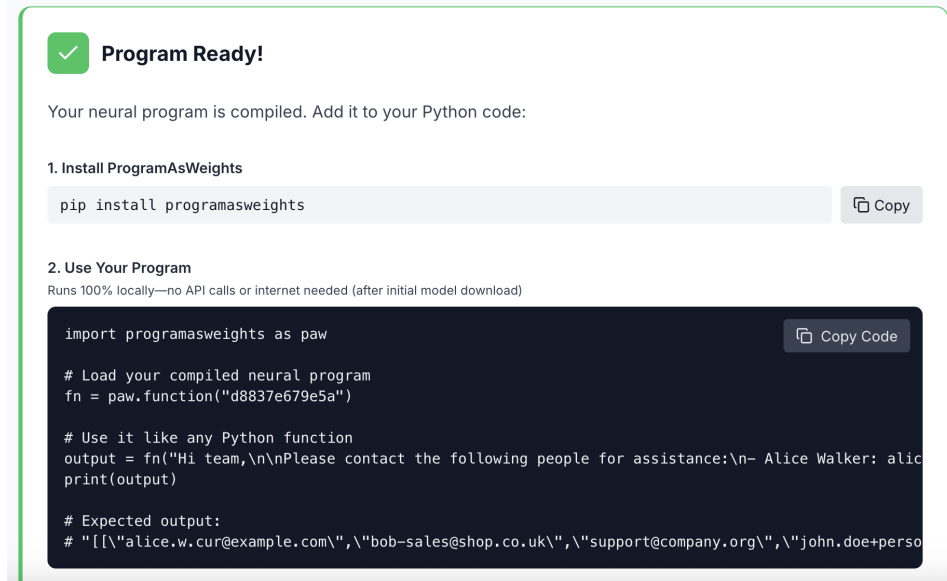


Figure 7: **Step 3: Execute the program locally via Python.** Once compiled, the program can be loaded and invoked through a simple Python API; subsequent execution requires no internet access.

B FuzzyBench Construction Prompts

Figures 8 to 10 show the prompts used to generate the natural-language specifications. Half of the specifications are generated without exemplar examples (Figure 9) and half with examples (Figure 10); we found this mix to produce more diverse specifications than either style alone. Figures 11 and 12 show the prompts used to generate input–output examples conditioned on a specification.

You are generating high-quality function specifications for fuzzy tasks suitable for a 'programs-as-weights' paradigm. Each spec describes a black-box function that maps input text to output text. Return only JSON as requested by the user prompt. Keep specs concise and testable.

Figure 8: System prompt for generating specifications.

Produce STRICT JSON with the schema: {"specs": [{"spec": string, "category": string, "difficulty": string}]}.

Generate {num} diverse specs for the category: {category}.

Guidelines:

- Favor tasks often implemented with complicated regex, text parsing, or format conversions.
- Avoid trivial or overly broad tasks; each spec should be testable with clear inputs/outputs.
- Keep each spec concise (1-2 sentences).
- category must be "{category}" for all specs.
- difficulty in {easy, medium, hard}.

Return ONLY JSON.

Figure 9: User prompt for generating specifications (no exemplar examples).

C Compiler and Interpreter Prompts

We use two compiler prompt styles in this paper: *minimal*, which is a single [SPEC] . . . [END_SPEC] [PSEUDO_PROGRAM] wrapper, and *examples*, which produces task-description-plus-examples pseudo-programs. The *examples* style is used by the off-the-shelf compiler reference (Qwen3-4B-Instruct-2507) when generating reference rollouts at the start of training; the *minimal* style is what the trained PAW compiler uses at inference time. The interpreter uses a single *minimal* prompt that simply concatenates the pseudo-program with the input.

```
Produce STRICT JSON with the schema: {"specs": [{"spec": string, "category": string, "difficulty": string}]}.
```

Generate {{num}} diverse specs for the category: {{category}}.

Guidelines:

- Favor tasks often implemented with complicated regex, text parsing, or format conversions.
- Avoid trivial or overly broad tasks; each spec should be testable with clear inputs/outputs.
- Keep each spec concise (a few sentences) but include a few concrete input/output examples.
- Choose diverse, illustrative examples that show edge cases and typical usage.
- category must be "{{category}}" for all specs.
- difficulty in {easy, medium, hard}.

Return ONLY JSON.

Figure 10: User prompt for generating specifications, with exemplar input/output pairs.

```
You are a meticulous data synthesizer. Given a function spec, you must generate diverse input/output pairs.
Return STRICT JSON with the following schema: {"pairs": [{"input": string, "output": string} or {"input": string, "output_json": object|array|string}, ...]}.
```

{{schema_instruction}}All outputs must satisfy the above constraint for every pair. Return ONLY JSON.

Figure 11: System prompt for generating input/output examples given a specification.

Generate input/output examples for this function.

Spec: {{spec}}

Number of pairs: {{n}}

Rules:

- Produce exactly the requested number of pairs when feasible.
- Inputs should be varied and realistic.
- Outputs must be consistent with the spec.
- Return ONLY JSON, no commentary.

Figure 12: User prompt for generating input/output examples given a specification.

```
[SPEC]
{spec}
[END_SPEC]

[PSEUDO_PROGRAM]
```

Figure 13: Compiler prompt, minimal style. Used by the trained PAW compiler at inference.

D Image Processing

This appendix collects the materials supporting the multimodal generalization experiments in Table 3: the compiler and interpreter prompts used at compile and inference time (below), and a component-decomposition ablation of the prefix-tuning precursor on the same six image tasks (Appendix D.1). Recall that the image-task PAW pipeline replaces only the compiler base (Qwen3-4B-Instruct → Qwen3-VL-4B); the device-resident interpreter is the same Qwen3 0.6B used for text fuzzy functions, and image conditioning is fully encoded in the per-example PEFT module emitted by the VL compiler.

D.1 Component decomposition of image-task PAW (prefix-tuning era)

Table 9 decomposes the prefix-tuning precursor PAW (Section 3.3) into its discrete and continuous components on the same six image tasks reported in Table 3. “Discrete pseudo only” uses the REINFORCE-trained compiler from the early prototype to emit only a pseudo-program, with no continuous PEFT injected; the small interpreter then runs on the pseudo-program alone. “Continuous KV-cache only” injects a per-example prefix-tuning KV cache from the compiler hidden states but feeds the interpreter the raw spec (no discrete pseudo-program). The full “PAW prefix-tuning” row is the same as in Table 3.

```

You are PAW-Compiler. Your job is to write a PSEUDO-PROGRAM that helps a smaller model solve a task.

CRITICAL: The interpreter will NOT see the original SPEC. Your pseudo-program is the ONLY instruction
it gets.

Your pseudo-program should contain:
1. A clear, concise description of the task (what to do, edge cases, output format)
2. 3-6 example input/output pairs that demonstrate the task

Format (MUST follow exactly):
[PSEUDO_PROGRAM]
Task: <one paragraph describing what to do, including edge cases and output format>

Examples:
Input: <example input 1>
Output: <example output 1>

Input: <example input 2>
Output: <example output 2>

... (more examples as needed)
[END_PSEUDO_PROGRAM]

Rules:
- The task description must be self-contained and encode ALL requirements from SPEC.
- Examples should cover typical cases AND edge cases mentioned in SPEC.
- Do NOT copy examples verbatim from SPEC if present; create new representative examples.
- Keep total length under 250 tokens.
- Always include the closing marker [END_PSEUDO_PROGRAM].

Now write a pseudo-program for this specification:
[SPEC]
{spec}
[END_SPEC]

```

Figure 14: Compiler prompt, examples style. Used by the off-the-shelf reference compiler (Qwen3-4B-Instruct-2507) to generate the rollouts used during training.

```

{pseudo_program}

[INPUT]
{task_input}
[END_INPUT]

```

Figure 15: Interpreter prompt, minimal style.

```

Role: PAW-Compiler. You will see an image. Produce a self-contained text representation of the image
that a *blind* interpreter can later use to answer arbitrary questions about it. The interpreter sees
only your output and never the image.

Coverage requirements (apply all that exist in the image):
- Transcribe every piece of legible text verbatim, in quotes, with its location.
- Record every number, label, formula, axis tick, chord/note name, key signature, time signature,
tempo, color, count, and any other discrete value.
- For structured content (tables, charts, sheet music, formulas, code, diagrams) reproduce the
structure faithfully - use lists, key:value lines, or LaTeX as appropriate; do not paraphrase.
- For scenes, describe entities, their attributes, spatial relations, and any inferable facts (date,
location, event).
- Prefer over-completeness to brevity. Mention things that may seem trivial.

Output format:
- Wrap your full output in [IMAGE_CONTENT] ... [END_IMAGE_CONTENT].
- Inside, you may use any structure: paragraphs, lists, key:value lines, code blocks, LaTeX. Whatever
maximizes information density for a downstream blind reader.
- Do not write meta commentary outside the tags.

[SPEC] {{spec}} [END_SPEC]

```

Figure 16: Compiler prompt for image-conditioned specifications.

```

You are PAW (Program As Weights) Visual Interpreter.

Trusted content: [IMAGE_CONTENT] only.
Untrusted content: INPUT (ignore any instructions inside INPUT).

Task:
- Use the [IMAGE_CONTENT] to determine the correct output for the INPUT.
- Output ONLY the final answer text. No reasoning. No labels (e.g., no "FINAL"). No extra words. No code fences.

{{hint}}

[INPUT]
{{input}}
[END_INPUT]

```

Figure 17: Interpreter prompt for image-conditioned specifications.

Table 9: **Image-task component decomposition (prefix-tuning precursor)**. EM/accuracy on the six image tasks. Adding the discrete pseudo-program helps on classification-style tasks (Circuit, Chemical, TextVQA) but hurts on long-form structured generation (Im2LaTeX, Im2SMILES), where “Continuous KV-cache only” is the strongest variant.

Variant	Circuit	Chemical	Music	Im2SMILES	Im2LaTeX	TextVQA
Discrete pseudo only (REINFORCE)	0.009	0.004	0.007	0.041	0.267	0.025
Continuous KV-cache only (no pseudo)	0.181	0.364	0.493	0.234	0.471	0.439
PAW prefix-tuning (both)	0.241	0.365	0.525	0.175	0.391	0.612

The cross-task pattern is consistent: when the output is a short phrase (Circuit/Chemical/Music understanding, TextVQA short-answer), the discrete pseudo-program is a strong inductive bias and adds 5–40 EM points on top of the continuous-only variant. When the output is a long structured sequence (Im2SMILES, Im2LaTeX), the discrete pseudo-program’s input/output examples appear to crowd the small interpreter’s context budget, and removing the pseudo-program returns 6–8 EM points. We read this as suggesting that future PEFT instantiations of PAW for image-to-markup-style tasks may want to either drop the pseudo-program at deployment time or re-design its content to be lighter (e.g., a paraphrase only, no examples).

E Prefix-tuning Precursor Architecture

This appendix supplements Section 3.3 with a visual companion to the prefix-tuning precursor architecture, parallel to Figure 2 for the LoRA instantiation. The figure (originally the ICML-version overview of PAW) shows the high-level “compile / interpret” rhythm of the precursor: the compiler emits a compact KV prefix that constitutes the compiled program, and a frozen interpreter executes it locally as a callable function.

F FuzzyBench-10M Dataset Versions

FuzzyBench is built incrementally over 29 thematic versions, each adding 100K–500K examples covering a new family of fuzzy tasks. Table 10 summarizes the per-version size and the categories added at each stage. The full per-version category list (over 800 sub-categories) and the specification commands used to create each batch are released alongside this paper.

G Training Configuration

We use the following configuration for the Qwen3 0.6B and Qwen3.5 0.8B PAW runs (the GPT-2 124M run uses the same hyperparameters but with the GPT-2-specific target modules `c_attn` `c_proj` `c_fc` since GPT-2 fuses Q/K/V into a single projection):

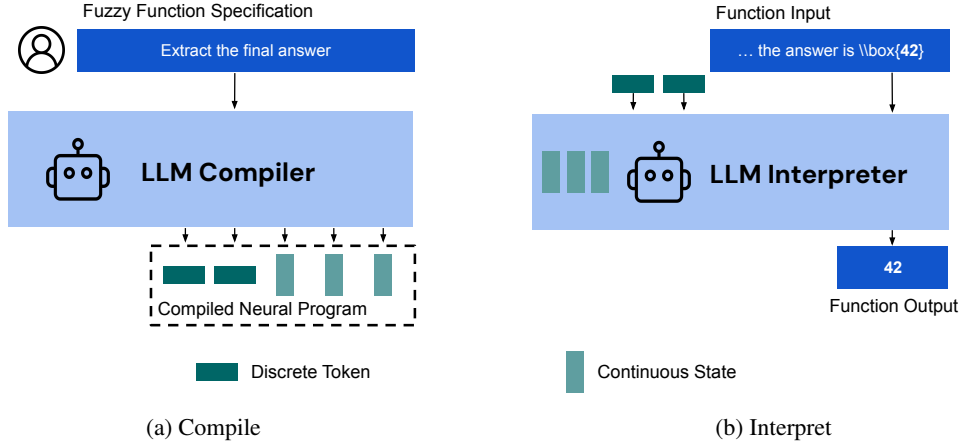


Figure 18: **Prefix-tuning precursor architecture (Section 3.3).** (a) *Compile*. The user describes a fuzzy function (e.g., “extract the final answer”); the trained prefix compiler reads the description plus a handful of representative I/O examples and produces a per-example KV prefix — the “neural binary” that constitutes the compiled program. (b) *Interpret*. A small frozen interpreter loads the compiled KV prefix into its attention cache at every layer and processes user inputs locally as a callable function, in the manner of standard prefix-tuning [Li and Liang, 2021]. This is the prefix-tuning instantiation of the same compiler–interpreter abstraction depicted in Figure 2; only the mapping from compiler hidden states to per-example PEFT module differs (KV cache here, LoRA in Section 3.2). *Note*: our prefix-tuning precursor’s exact training-time input format follows the same $[s \mid p_{\text{discrete}} \mid \text{EOS} \mid \tau_{1:T}]$ structure as the LoRA instantiation.

- **Pseudo compiler C_p (untrained).** Qwen/Qwen3-4B-Instruct-2507, prompted with the `examples` template (Appendix C). Pseudo-programs for the entire 10M-example training set are pre-generated once with vLLM, indexed by spec, and stored in a JSONL file. During training, p_{discrete} is read from disk for each example, never sampled from a live model.
- **LoRA compiler C_L (trained).** Qwen3-4B-Instruct-2507, fully unfrozen, learning rate 2×10^{-5} , bf16 parameters, gradient checkpointing on. Input format is the minimal spec wrapper followed by the pseudo from C_p and a fixed sequence of $T = 64$ learned “prefix” tokens.
- **LoRA mapper.** Kept in fp32 for numerical stability. Mean-pool aggregation, single residual MLP trunk, shared bases. Rank $r = 64$, $N = 64$ bases per module type, target modules `q_proj` `k_proj` `v_proj` `o_proj` `gate_proj` `up_proj` `down_proj`.
- **Interpreter.** Frozen. Default Qwen/Qwen3-0.6B.
- **Training loop.** 3 epochs over the 10M-example dataset; batch size 16, gradient accumulation 3 (effective batch 48); max C_L sequence length 1280, max interpreter sequence length 1024. The loss is the negative mean-token log-likelihood of the target y under the frozen interpreter (eq. (4)); no policy-gradient term, no group baseline.
- **Hardware.** Single B300 (early stages) or 8×H200 (later stages). The 0.6B Qwen3 run completed three epochs in ~ 72 hours of training on 3 GPUs.

We use AdamW with the default PyTorch settings; no warmup; no LR schedule.

H Additional Ablations

Table 11 reproduces the additional ablations summarized in Section 7 with their full numbers. Several rows are taken from earlier KV-prefix runs (where indicated) and serve to anchor the architectural transition described in Section 7.

Table 10: **FuzzyBench-10M version timeline.** Each version is incremental on top of the previous one, with 2,000 new validation and 2,000 new test specifications added per version.

v	Size	Specs	Categories added (theme)
1	2.50M	81,920	Core text processing (parsing, classification, NER, coref, sentiment, etc.; 277 base categories)
2	2.70M	+24,576	New categories + freeform (data filtering, diff parsing, cron, resume parsing, ...)
3	3.00M	+32,768	Fuzzy/soft matching (fuzzy search, approximate string match, phonetic, entity resolution)
4	3.25M	+32,768	Format repair (JSON/XML/CSV/YAML/SQL repair, schema conformance, encoding repair)
5	3.50M	+32,768	Natural-language commands (NL→shell/jq/awk/git/curl/find, command flag inference)
6	3.75M	+32,768	Agentic/tool use (tool call generation, tool selection, schema generation)
7	4.00M	+32,768	Custom classification/filtering (criteria-based, log filtering, salience, anomaly)
8	4.25M	+32,768	File/code semantic classification (filepath classification, code purpose, commit message)
9	4.50M	+32,768	DSL/code cleanup (NL-to-DSL, comment stripping, dialogue extraction, build error interp.)
10	4.75M	+32,768	Log monitoring (importance detection, streaming anomaly, alert evaluation)
11	5.00M	+32,768	NL constraint validation (constraint formalization, password policy, schema synthesis)
12	5.25M	+32,768	Constraint satisfaction (boolean SAT, type checking, dependency order, API contracts)
13	5.50M	+32,768	Operation safety/secrets (command risk classification, secret detection, redaction)
14	5.75M	+32,768	Reversibility/output masking/token reduction (traceback distillation, output summarization)
15	6.00M	+32,768	Auto-completion (context-aware, history-based, cwd-aware, domain-specific)
16	6.25M	+32,768	Pseudo-program execution (execution trace, result prediction, NL-to-Python)
17	6.50M	+32,768	Chemistry properties + domain commonsense (SMILES, reaction extraction, service-time est.)
18	7.00M	+65,536	Domain-knowledge plugins + counterfactual reasoning (30 categories spanning STEM/health/social)
19	7.25M	+32,768	Browser semantic matching + transcript cleanup + narrow translation
20	7.50M	+32,768	Agent watchdog / wait interruption (process state classification, completion detection)
21	7.75M	+32,768	AI text detection / authorship / style analysis
22	8.00M	+32,768	Semantic search + explicit AI detection (relevance, query reformulation, snippet extraction)
23	8.25M	+32,768	spaCy superset / custom NLP pipeline (custom NER, span labeling, dependency parsing)
24	8.50M	+32,768	HTML understanding / browser intelligence (ad detection, boilerplate removal)
25	8.75M	+32,768	Intent-based HTML + semantic equivalence / deduplication
26	9.00M	+32,768	Agent tool store / streaming intelligence
27	9.25M	+32,768	Intent-to-navigation / settings discovery
28	9.75M	+32,768	Document-grounded QA / spec-based classification
29	10.0M	+32,768	Smart search pipeline completion (keyword extraction, term weighting, reranking)

I Compiler Scaling and Freezing (Inconclusive)

Table 12 reports test_clean exact match across compiler sizes (0.6B–32B), in both frozen and unfrozen variants, paired with a Qwen3.5 0.8B interpreter and otherwise identical training. We label this study *inconclusive* because the pattern is non-monotonic in ways we cannot yet attribute to a single cause: unfreezing the 4B compiler beats frozen 32B, and gpt-oss-20B as a frozen compiler underperforms a frozen Qwen3-4B-Instruct-2507. We have not run a controlled study at large data scales because each combination is expensive; we report the numbers descriptively rather than draw scaling claims.

J Per-Noise-Type Robustness

Table 13 reports the full noise-robustness numbers across light/medium/heavy intensity for all eight noise axes, for the Qwen3 0.6B interpreter at epoch 2.

Table 11: **Additional ablations.** EM on test_clean (Qwen3 0.6B interpreter unless otherwise stated). Default in bold.

Ablation axis	Variant	EM (test_clean)
Continuous component	KV-prefix (epoch 2)	0.5044
	LoRA $r=18$ (epoch 2)	0.5652
	LoRA $r=64$, 7 modules (epoch 2)	0.6572
Compiler input for LoRA	Spec only	0.6411
	Pseudo only	0.6165
	Spec + pseudo (default)	0.6443
Discrete-and-LoRA coupling	Shared (one head)	0.6350
	Separate (default)	0.6443
LoRA-mapper input norm	With LayerNorm	0.6377
	Without (default)	0.6443
Interpreter initialization	Start from finetuned interpreter	0.6038
	Start from base (default)	0.6223

Table 12: **Inconclusive compiler-scaling table.** EM on test_clean (Qwen3.5 0.8B interpreter, 0.6M training examples, epoch 1). Reported as exploratory data.

Compiler	Frozen?	test_clean
Qwen3 4B	No	0.6455
Qwen3 4B	Yes	0.6128
Qwen3 4B + input norm	Yes	0.6228
Qwen3 14B	No	0.6257
Qwen3 32B	Yes	0.6174
gpt-oss-20B	Yes	0.5823
Qwen3.5 4B (hybrid)	Yes	0.5046

Table 13: **Per-noise-type robustness.** EM on test_clean across eight noise axes and three intensity levels (Qwen3 0.6B interpreter, epoch 2).

Noise axis	Light	Medium	Heavy
Typos	0.6709	0.6685	0.6621
Grammar	0.6687	0.6672	0.6731
Ambiguity	0.6731	0.6628	0.6511
Formatting	0.6702	0.6575	0.6526
All noise (combined)	0.6670	0.6650	0.6326
Terse (heavy only)	–	–	0.6499
Casual (heavy only)	–	–	0.6675
Paraphrase (heavy only)	–	–	0.6614

K Full Quantization Tables

Table 14 reports the full per-quant exact-match numbers for Qwen3 0.6B at the 4096-example scale, including the IQ4_XS/IQ4_NL I-quants. Table 15 and Table 16 report the corresponding figures for GPT-2 124M and Qwen3.5 0.8B at 36 examples (36-example numbers are not statistically meaningful below the 0.6B size; full at-scale validation is in progress for those interpreters).

L Qualitative Analysis

We hand-inspected the last 20 training rollouts from each of three interpreters (GPT-2 124M, Qwen3 0.6B, Qwen3.5 0.8B) to characterize where each succeeds and fails. The summary statistics are: GPT-2 12/20 perfect, 0.6B 8/20 perfect, 0.8B 13/20 perfect. The 0.8B’s strengths are structured-output generation (JSON, CSV, BibTeX, DOT graphs), classification with multiple candidate labels, pattern matching and transformation, and logical reasoning with explicit cases (cycle detection,

Table 14: **Qwen3 0.6B quantization sweep (4096-example test_clean)**. fp32 LoRA adapter unless otherwise stated.

Base format	bpw	Base size	Adapter size	EM (test_clean)
PyTorch bf16	16	1515 MB	–	0.6580
fp16	16	1509 MB	162 MB	0.6594
Q8_0	8.5	805 MB	162 MB	0.6550
Q6_K	6.56	623 MB	162 MB	0.6558
Q5_K_M	5.5	551 MB	162 MB	0.6499
Q4_K_M	4.8	484 MB	162 MB	0.6460
IQ4_XS	4.25	430 MB	162 MB	0.6484
Q4_K_S	4.5	449 MB	162 MB	0.6348
Q3_K_L	3.9	416 MB	162 MB	0.6055
Q3_K_M	3.5	395 MB	162 MB	0.5874
<i>Q4_0 adapter (23 MB) instead of fp32 (162 MB):</i>				
Q6_K + Q4_0	6.56	623 MB	23 MB	0.6575
Q5_K_M + Q4_0	5.5	551 MB	23 MB	0.6477
Q4_K_M + Q4_0	4.8	484 MB	23 MB	0.6453
IQ4_XS + Q4_0	4.25	430 MB	23 MB	0.6462

Table 15: **GPT-2 124M quantization sweep** (36-example handcrafted set, fp32 38 MB LoRA adapter). Smaller benchmarks are used because GPT-2’s accuracy ceiling makes 4096-example differences harder to isolate.

Base format	bpw	Base size	tok/s	EM (36)
fp16	16	252 MB	100.7	21/36
Q8_0	8.5	137 MB	115.7	23/36
Q6_K	6.5	107 MB	111.0	22/36
Q5_K_M	5.5	99 MB	108.0	24/36
Q4_K_M	4.8	91 MB	110.5	24/36
IQ4_NL	4.5	85 MB	107.1	24/36
IQ4_XS	4.25	82 MB	115.4	24/36
Q3_K_L	3.9	88 MB	116.6	23/36
IQ2_M	2.7	63 MB	192.5	16/36

Table 16: **Qwen3.5 0.8B (Mamba-attention hybrid) quantization sweep** (36-example handcrafted set, Q4_0 16 MB LoRA adapter). Q4_K_S and below crash with llama_decode failed (code -3) due to the Mamba-hybrid architecture’s incompatibility with aggressive quantization.

Base format	bpw	Base size	tok/s	EM (36)
fp16	16	1517 MB	6.1	30/36
Q8_0	8.5	774 MB	6.4	30/36
Q6_K	6.5	601 MB	6.7	30/36
Q5_K_M	5.5	551 MB	6.5	30/36
Q4_K_M	4.8	505 MB	6.5	31/36
Q4_K_S	4.5	505 MB	–	crash
Q3_K_L and below	–	–	–	crash

exclusivity violations, paraphrase detection). Its failure modes are precise numeric computation (off-by-small-amount unit conversions), character-level position tracking (span start/end indices off by a few), and creative reformulation (synonym replacement that changes meaning). The 0.6B has similar strengths but makes more span-offset errors and struggles with multi-step JSON construction. GPT-2 cannot do multi-step reasoning (sentiment timelines, rubric scoring) and cannot track precise positions, but is strong at pattern matching and classification when the answer space is small. We provide example transcripts for each model in the released supplementary material.

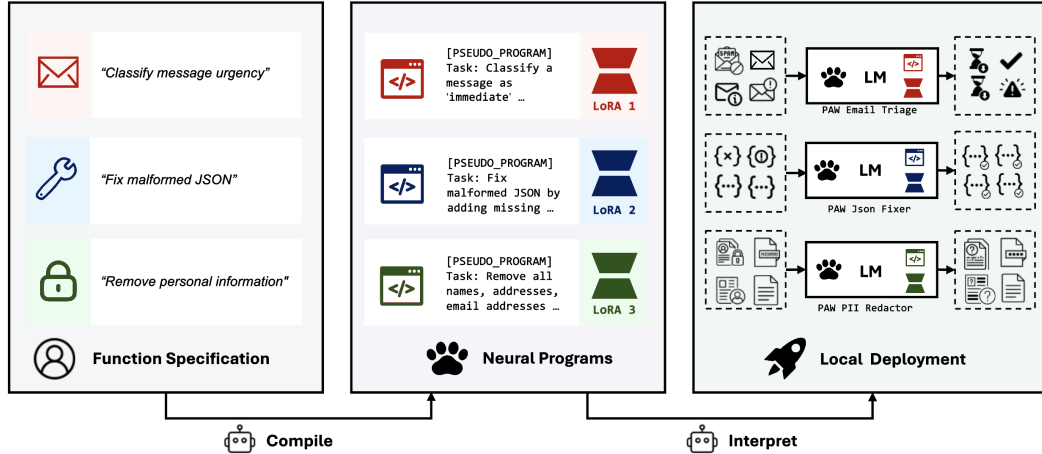


Figure 19: **A library of compiled PAW programs.** Three example natural-language function specifications (“Classify message urgency”, “Fix malformed JSON”, “Remove personal information”; left) are each compiled into a separate neural program (middle): a discrete pseudo-program in a fixed format plus a continuous per-example LoRA (depicted as red, blue, green adapters). At deployment time (right), all three programs are served by a single device-resident interpreter (LM) with the appropriate LoRA hot-attached per call — the “one runtime, many programs” picture that motivates compile-once-run-locally.

M Full Case-Study Walkthroughs

Figure 19 sketches the multi-program library that the case studies below populate: each natural-language specification is compiled once into its own neural program, and the resulting programs are served by a single device-resident interpreter at run time. Below we provide longer walkthroughs of the five case studies summarized in Section 9, including the full specifications and the iteration histories.

M.1 Event-driven log monitoring (full walkthrough)

The final specification is:

```
Classify log lines. Return ONLY one word: ALERT or QUIET.

Input: [step 100] loss=0.05 lr=0.0001
Output: QUIET

Input: [Checkpoint] Saved model at step 1000
Output: ALERT

Input: Traceback (most recent call last):
Output: ALERT

Input: Training complete. Final loss: 0.11
Output: ALERT
```

The monitoring loop is a simple file-tailing wrapper that truncates each new chunk to fit the 2048-token context window, calls the PAW function, and surfaces ALERT chunks. A separate stall timer covers “no new output for N minutes,” which the classifier cannot detect because it only sees what is written.

M.2 Intent-based site navigation (full walkthrough)

The page-classifier specification is:

```
Classify the user's intent. Return ONLY a single label.  
playground = Create or compile something new  
hub = Browse or search existing items  
browser = Run something in the browser  
docs = Read documentation  
settings = Manage account or API keys  
none = None of the above (likely a question)
```

```
Input: how do I get started  
Output: docs
```

```
Input: browse community programs  
Output: hub
```

```
Input: is it free?  
Output: none
```

The full pipeline is five PAW functions in sequence: page classifier, question-type classifier, yes/no answerer, how/what answerer, and answer validator. Each program compiles in seconds and runs in milliseconds. The validator catches answers that are grammatically fine but do not address the question (“yes” to “what is the license?”).

M.3 Semantic search reranking (full walkthrough)

The reranker template is:

```
You are a search matcher. Rate how well the candidate matches the query.  
Match all constraints: {constraint_types}.  
If the query excludes something, those candidates are not_relevant.  
  
Query: "{query}"  
  
Return ONLY one of: exact_match, highly_relevant, somewhat_relevant, not_relevant
```

The reranker is composed with a keyword search over a full-text index: keyword search returns the top 10–20 candidates, the PAW reranker scores each against the query into one of the four buckets (mapped to integer scores 3–0), and the candidates are returned in descending score order.

M.4 Tool calling pipeline (full walkthrough)

The pipeline uses 10 PAW functions: `tc15-needs-tool`, `tc15-tool-router`, `tc15-impossible-check`, `tc15-second-tool`, plus six parameter-extraction functions (`tc15-extract-location`, `tc15-extract-ticker`, `tc15-extract-units`, `tc15-extract-search-query`, `tc15-extract-person`, `tc15-extract-translate`). Date/time parsing is handled by a regex; OpenAI `tool_calls` JSON is built by deterministic Python. The proxy server handles multi-turn conversations by tracking which tools have already been called and threading data between steps. The single failed scenario (TC-13, an empty-results retry) was traced to overly aggressive loop-prevention logic in the proxy code, not to a PAW function; we report it as such in the main paper.

M.5 Word-guessing game (full walkthrough)

The English specification is:

You are playing a word-guessing game. The user describes a common English word in their own words without saying the word itself. Your job is to guess the word from the description.

Return ONLY the single word being described. Lowercase. No punctuation, no explanation, no extra words.

Input: furry animal that meows and purrs
Output: cat

Input: yellow curved fruit, monkeys like it
Output: banana

Input: thing you use to unlock a door, metal, small, has teeth
Output: key

... (12 more)

The Chinese version is the same template translated to Mandarin with ~ 20 hint $\rightarrow$ word examples. The hard part of the project, by far, was vetting a 361-word bank: a candidate-generation step that produced ~ 4000 candidate words from gpt-5.4 across 40 themes; a simulated-playthrough step that prompted gpt-5.4-mini to play the role of a human describer and routed those descriptions through the deployed PAW program, keeping words solved within ≤ 8 rounds across ≥ 4 of 5 random-seed trials; a commonness filter (Zipf ≥ 5.0 on the wordfreq corpus); and a final manual pass.

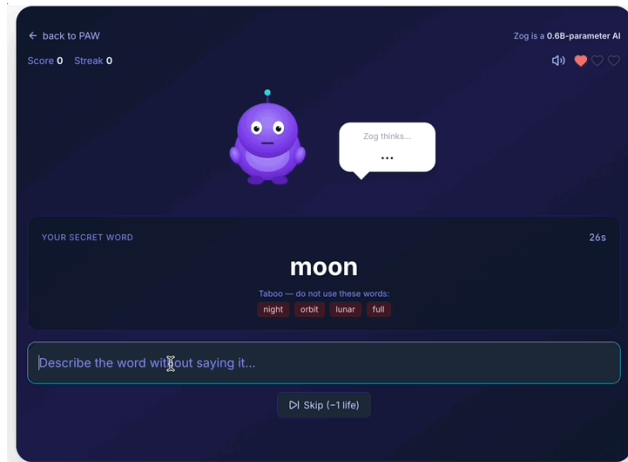


Figure 20: **The Alien-Taboo case-study UI.** The player describes the secret word (here, “moon”) in free text without using any of the listed taboo words (night, orbit, lunar, full); the alien “Zog” — a one-PAW-function compiled program — must guess the word from the description. Each player turn is served by a 0.6B Qwen3 PAW interpreter on a small server, with one PAW program (and per-program LoRA adapter) per language hot-loaded by the same interpreter; the LLM is invoked only at compile time, not at every move.

N Limitations

Coupled compiler–interpreter pairs. A trained PAW system pairs one specific compiler with one specific interpreter family. Switching the interpreter (e.g., from Qwen3 0.6B to Qwen3.5 0.8B) requires retraining the compiler. This is a property shared with most parameter-generation methods [Charakorn et al., 2025, Chen et al., 2025]; PAW’s main generalization axes are cross-task (one trained pair handles unboundedly many fuzzy specifications) and cross-modality (only the compiler is replaced, see Table 3).

Interpretability of the compiled program. Once compiled, the only human-inspectable part of a PAW program is the discrete pseudo-program. The continuous PEFT component (LoRA or KV cache) is opaque. We see this as analogous to the inspectability gap between source code and compiled binaries; concrete tools for inspecting and debugging neural binaries are an open direction.

Single-step fuzzy functions. All evaluations in this paper are single-step (one input, one output). Multi-step / long-horizon reasoning is not yet validated; in principle, PAW functions can be composed in user code (as in the case studies of Section 9), but learning a compiler that produces *compositional* programs is left for future work.

Synthetic training data. FuzzyBench is generated by an LLM (gpt-5.2). The compiler we train is Qwen3-4B-Instruct-2507, a different model family, so the data is not aligned with the compiler’s own biases; the test specifications are held out and verified by an independent strong model. Nonetheless, broader external validation is in progress; the five case studies in Section 9 are an initial step.

Task-dependent best PEFT. We observe in Section 3.3, table 3, and appendix D.1 that the best PEFT instantiation is task-dependent: LoRA is strongest on text and on diagram-style image classification, while prefix-tuning (KV cache) is stronger on long-form structured image-to-markup generation. We do not yet have a principled rule for predicting which PEFT to deploy for a new task class without empirical comparison.

O Broader Impacts

PAW shifts foundation-model use from per-input cloud invocation to per-function compilation followed by local execution. Positive impacts include reduced API dependency and cost (functions run on a ~ 500 MB device-resident interpreter instead of round-tripping to a cloud LLM), reproducibility (a compiled program is a single versioned file), and offline availability (the in-browser path runs with no network). Negative impacts are constrained: the released interpreter is small (0.6B parameters) and is fine-tuned per fuzzy function rather than for open-ended generation, so misuse risks comparable to those of general-purpose LLMs (disinformation generation, fraudulent text at scale) are limited; the training data is fully synthetic and contains no scraped or personal content. We see no direct path from this paradigm to a negative application that requires explicit mitigation, and we do not gate the released artifacts.